



The **LIPI** Handbook

A programming language with Devanagari syntax — compiled to a bytecode VM in 100% pure Rust, running on desktop and in the browser.

Version 0.2 · Language guide, standard library, and Hindi tutorial



Contents

- 1. 1 · What is LIPI
- 2. 2 · Installation
- 3. 3 · Language basics
- 4. 4 · Control flow
- 5. 5 · Functions, closures & lambdas
- 6. 6 · Lists & dictionaries
- 7. 7 · Classes & objects
- 8. 8 · Error handling
- 9. 9 · The भारत standard library
- 10. 10 · Config & data formats
- 11. 11 · FFI, hardware & HTTPS — the no-limits tier
- 12. 12 · Tooling
- 13. 13 · परिचय — a tutorial in Hindi
- 14. 14 · Quick reference

1 • What is LIPI

LIPI (लिपि) is a general-purpose programming language whose keywords, error messages, and standard library are **Devanagari-native**. You write code in Hindi/Sanskrit vocabulary; the compiler turns it into bytecode for a stack virtual machine (the LVM), which then executes. The entire system is written in **100% pure Rust** with no external dependencies, and also compiles to WebAssembly to run in a browser.

Why it exists

Every English-keyword language adds a translation tax for a programmer who thinks in Hindi. LIPI removes it, and ships a standard library that already speaks Indian-context domains (Aadhaar, UPI, GST, IST) alongside a general toolkit.

The execution pipeline

```
source.swami
→ lexer      (Devanagari tokens, INDENT/DEDENT)
→ parser      (SOV grammar → AST)
→ compiler    (AST → LVM bytecode)
→ LVM         (stack VM → output)
```

The first program:

```
बताओ "नमस्ते दुनिया"
```

Output: नमस्ते दुनिया

2 • Installation

Platform	Command
Linux	<code>curl -fsSL ../install.sh sh</code>
macOS	<code>brew install <you>/tap/lipi</code> or the install script
Windows	<code>irm ../install.ps1 iex</code> or the .exe + install.bat
From source	<code>cargo build --release → target/release/lipi</code>

The CLI

Command	Does
<code>lipi foo.swami</code>	compile + run
<code>lipi build / run</code>	compile to .libc bytecode / execute it
<code>lipi test / fmt / lint</code>	run tests / format / lint
<code>lipi doc / profile / debug</code>	docs / profiler / step debugger
<code>lipi lsp / pkg / ide</code>	language server / package manager / browser IDE
<code>lipi</code>	REPL

3 · Language basics

Variables & printing

```
क है 5           | assignment |
अ, ब है 1, 2     # tuple unpacking
बताओ क + 1      # print with newline
लिखो "नाम: "     # print, no newline
बताओ ५ लाख     # Devanagari digits + lakh suffix
```

Output: 6 / नाम: / 500000

Operators

Kind	Operators
Arithmetic	+ - * / % // (floor div)
Compare	से अधिक (>) · से कम (<) · बराबर (==) · != · chainable 0 < x < 10
Boolean	और · या · नहीं
Bitwise	& ^ ~ << >>
Membership	x में_है सूची · x नहीं_है सूची

```
बताओ 7 // 2     # 3
बताओ 12 & 10     # 8
बताओ "नमस्ते {नाम}!" # string interpolation
```

4 • Control flow

Conditionals

```
यदि क से अधिक 10:
    बताओ "बड़ा"
अन्यथा यदि क बराबर 10:
    बताओ "बराबर"
अन्यथा:
    बताओ "छोटा"

परिणाम है यदि क से अधिक 5 तो "बड़ा" अन्यथा "छोटा" # ternary
```

Loops

```
5 बार करो:                # repeat N times
    बताओ "लूप"

i के लिए 10 में:          # for i in 0..9
    बताओ i

जब तक क से कम 10:       # while
    क है क + 1
    यदि क बराबर 5: अगला    # continue
    यदि क बराबर 8: बंद करो # break
```

5 • Functions, closures & lambdas

```
विधि जोड़ो(अ, ब):          # function; फल = return
    फल अ + ब
```

```
विधि नमस्ते(नाम="दुनिया"):  # default parameter
    फल "नमस्ते " + नाम
```

```
बताओ जोड़ो(ब=4, अ=3)      # keyword arguments
```

Output: 7

Closures & higher-order functions

```
विधि गुणक(न):
    फल लाम्डा(x): न * x    # captures न
दोगुना है गुणक(2)
बताओ दोगुना(7)
बताओ छानो([1,2,3,4,5,6], लाम्डा(x): x % 2 बराबर 0)
```

Output: 14 / [2, 4, 6]

मानचित्र (map), छानो (filter), मोड़ो (reduce), decorators, generators (उत्पन्न), and functools (स्मरण, आंशिक, संयोजित) are all built in.

6 · Lists & dictionaries

```
अंक है [10, 20, 30]
बताओ अंक[0]          # 10
बताओ अंक[1:3]        # slice → [20, 30]
बताओ अंक[::-1]       # reverse
अंक है अंक.जोड़ो(40)   # append (copy-on-write, reassign)
बताओ [x*x के लिए x अंक में यदि x से अधिक 15] # comprehension
```

Output: 10 / [20, 30] / [30, 20, 10] / [400, 900, 1600]

```
व्यक्ति है {"नाम": "राम", "आयु": 25}
बताओ व्यक्ति["नाम"]
व्यक्ति["शहर"] है "दिल्ली"
बताओ व्यक्ति.कुंजियाँ()
```

Output: राम / ["आयु", "नाम", "शहर"]

7 • Classes & objects

```
वर्ग व्यक्ति:
  विधि बनाओ(नाम, आयु):      # constructor
    यह.नाम है नाम
    यह.आयु है आयु
  विधि परिचय():
    बताओ यह.नाम + ", आयु " + यह.आयु

वर्ग छात्र(व्यक्ति):              # inheritance
  विधि बनाओ(नाम, आयु, कक्षा):
    यह.नाम है नाम
    यह.आयु है आयु
    यह.कक्षा है कक्षा

प है व्यक्ति("राम", 30)
प.परिचय()
```

Output: राम, आयु 30

Also supported: static methods (साझा विधि), abstract classes (सार वर्ग), records (अभिलेख), properties (getters/setters), operator overloading, and context managers (साथ ... के_रूप_में).

8 • Error handling

```
वर्ग जाल_त्रुटि(त्रुटि):          # typed exception, inherits त्रुटि
    विधि बनाओ(संदेश):
        यह.संदेश है संदेश

कोशिश:
    फेंको जाल_त्रुटि("विफल")    # throw
पकड़ो जाल_त्रुटि ई:            # typed clause (matches subclasses)
    बताओ "पकड़ा: " + ई.संदेश
पकड़ो त्रुटि:                  # catch-all
    बताओ त्रुटि
```

Output: पकड़ा: विफल

Assertions use जाँचो; uncaught errors print a Hindi message with a line number and a call-stack trace.

9 · The भारत standard library

Import with `आयात भारत.<मॉड्यूल>`. A selection:

Module	Provides
पहचान / भुगतान / संख्या	Aadhaar/PAN/IFSC validation · UPI · GST, EMI, lakh/crore/rupee formatting
गणित	Ancient-Indian math: sine (ज्या), Fibonacci (विरहंक), combinatorics
json / csv / प्रतिमान	JSON, CSV (RFC 4180), regex engine
समय / कूट / बड़ी	datetime (IST) · sha256/md5/base64 · arbitrary-precision integers
संजाल / सर्वर / सूत्र	TCP sockets · HTTP server · OS threads
संग्रह / संपीडन	in-memory SQL · ZIP read/write
मात्रक / रेखीय / नियंत्रण	units & dimensional analysis · vectors/matrices/quaternions · PID + Kalman
दिशा / सुरक्षा / अंतराल	geodesy · Hamming ECC/CRC/TMR · interval arithmetic

```
आयात भारत.बड़ी
बताओ महा_भाज्य("50")    # 50! exactly

आयात भारत.मात्रक
बल है गुणा_मात्रा(मात्रा(2, "किग्रा"), मात्रा(9, "मीटर/सेकंडर"))
बताओ मात्रा_वाक्य(बल)

Output: 3041409320171337804361260816606476884437764156896051200...
18 न्यूटन
```

10 · Config & data formats

Parsers for the common config/data formats return native LIPI values (Dicts, Lists).

TOML / INI — भारत.टोमल

```
आयात भारत.टोमल
कोश है toml_पढ़ो("[सर्वर]\nपोर्ट = 8080")
बताओ कोश["सर्वर"]["पोर्ट"]
```

Output: 8080

YAML — भारत.यामल

```
आयात भारत.यामल
यामल_पढ़ो("सेवा:\n  पोर्ट: 8080\n  टैग:\n    - तेज")
```

Output: {"सेवा": {"पोर्ट": 8080, "टैग": ["तेज"]}}

XML / HTML — भारत.एक्सएमएल

```
आयात भारत.एक्सएमएल
ए है xml_पढ़ो("<पुस्तक id='1'><शीर्षक>गणित</शीर्षक></पुस्तक>")
बताओ ए["बच्चे"][0]["पाठ"]
```

Output: गणित

Command-line arguments — भारत.तर्कपार्स

```
आयात भारत.तर्कपार्स
तर्क_पार्स(["build", "--out=dist", "--fast"])
```

Output: {"out": "dist", "fast": सत्य, "_स्थिति": ["build"]}

11 · FFI, hardware & HTTPS — the no-limits tier

When the stdlib isn't enough, LIPI reaches native code and the network directly.

C FFI — भारत.बाह्य

Call any C-ABI function in any shared library. Cross-platform (LoadLibrary on Windows, dlopen on Linux/macOS). You can even hand a LIPI function back *as* a C callback — here C's qsort sorts using a LIPI comparator:

```
आयात भारत.बाह्य
पुस्तकालय है बाह्य_पुस्तकालय("msvcrt.dll")
सीएमपी है बाह्य_कॉलबैक(तुलना, "11:i")
बाह्य_बुलाओ(पुस्तकालय, "qsort", "1111:v", आधार, 5, 4, सीएमपी)
```

```
[5,2,8,1,9] → [1, 2, 5, 8, 9] (C called LIPI back)
```

Raw memory & MMIO — भारत.तंत्र

Bounds-checked buffers to build C structs, plus volatile कच्चा_पढ़ो/लिखो for memory-mapped hardware registers.

HTTPS / TLS — भारत.सुरक्षित

```
आयात भारत.सुरक्षित
प है https_पाओ("https://api.github.com/zen", {"User-Agent":"LIPI"})
बताओ प["स्थिति"]
```

```
Output: 200
```

With real HTTPS, LIPI can call LLM APIs, payment gateways, and cloud services. See `examples/llm_client.swami` for a Claude Messages-API wrapper.

12 • Tooling

Tool	Command
Test framework	<code>lipi test</code> — runs परीक्षण blocks in isolated VMs
Formatter	<code>lipi fmt</code> — behavior-preserving, idempotent
Linters	<code>lipi lint</code> — unused/undefined variables
Type checker	<code>lipi check</code> — optional gradual type hints
Doc generator	<code>lipi doc</code> — Markdown from signatures + comments
Profiler	<code>lipi profile</code> — opcode/function report (+ SVG flame graph)
Debugger	<code>lipi debug</code> — line breakpoints, step, inspect
Language server	<code>lipi lsp</code> — diagnostics, hover, completion (Neovim/VSCoDe)
Package manager	<code>lipi pkg</code> — <code>lipi.toml</code> + git-native install
Browser IDE	<code>lipi ide</code> — Monaco-based LIPI Studio

A test block

```
परीक्षण "जोड़ो सही है":  
  जाँचो जोड़ो(2, 3) बराबर 5, "जोड़ गलत"
```

13 · परिचय — एक हिंदी ट्यूटोरियल

यह खंड पूरी तरह हिंदी में है — बिना किसी पूर्व अनुभव के LIPi सीखने के लिए।

पाठ १ — पहला कार्यक्रम

बताओ का अर्थ है "स्क्रीन पर दिखाओ"। इसे चलाओ:

```
बताओ "मेरा पहला कार्यक्रम"
```

परिणाम: मेरा पहला कार्यक्रम

पाठ २ — चर (variables)

है का अर्थ है "मान रखो"। किसी नाम में संख्या या शब्द रखा जा सकता है।

```
आयु है 25  
नाम है "सीता"  
बताओ नाम + " की आयु " + आयु
```

परिणाम: सीता की आयु 25

पाठ ३ — निर्णय (if/else)

```
अंक है 72  
यदि अंक से अधिक 33:  
    बताओ "उत्तीर्ण"  
अन्यथा:  
    बताओ "अनुत्तीर्ण"
```

परिणाम: उत्तीर्ण

पाठ ४ — दोहराव (loops)

```
i के लिए 5 में:  
    बताओ i * i
```

परिणाम: 0 1 4 9 16

पाठ ५ — विधि (function)

```
विधि बड़ा(अ, ब):  
    यदि अ से अधिक ब:  
        फल अ  
    फल ब  
बताओ बड़ा(12, 7)
```

अभ्यास

एक विधि जीएसटी बनाओ जो राशि लेकर 18% कर जोड़कर लौटाए। संकेत: फल राशि + राशि * 18 // 100।

14 • Quick reference

Concept	Keyword	Concept	Keyword
print / inline	बताओ / लिखो	assign	है
if / elif / else	यदि / अन्यथा यदि / अन्यथा	ternary	... तो ... अन्यथा
for / while	के लिए ... में / जब तक	repeat N	N बार करो
break / continue	बंद करो / अगला	function / return	विधि / फल
lambda	लाम्ब्डा	class / self	वर्ग / यह
try / catch / throw	कोशिश / पकड़ो / फेंको	assert	जाँचो
and / or / not	और / या / नहीं	import	आयात
true / false / nil	सत्य / असत्य / शून्य	global / const	वैश्विक / स्थिर
yield	उत्पन्न	match	मिलाओ

Constraints to remember

- फल (return), है (assign), and वर्ग (class) are keywords — not usable as names.
- Lists/dicts are copy-on-write: mutating methods return a new value — reassign it.
- Comments are single-line: `I ... I` or `#`. No hex literals.

This handbook is generated from LIPI's own documentation. Every example was run on the reference implementation; outputs are real. Source: the LIPI repository, docs/handbook/.